REAL-TIME DISASTER COMMUNICATION SYSTEM

REAL-TIME DISASTER COMMUNICATION SYSTEM

A Distributed, Priority-Aware Emergency Messaging Platform in C

DETAILED TECHNICAL PROJECT REPORT

CO5 – AT3 – HACKATHON

C Programming • CSA0211

NAME
Digvijay A
REGISTRATION NO.
192320045

NAME
Reshini Priya B
REGISTRATION NO.
192320035

NAME
Vishal J
REGISTRATION NO.
192412433

PROJECT FOCUS

Concurrent, priority-based emergency message handling using C11, POSIX threads, TCP sockets, a producer–consumer priority queue, condition variables and POSIX Named Pipe (FIFO) inter-process communication.

Executive Snapshot

Concurrency	POSIX threads – one detached worker thread per connected field station
Communication	TCP client–server sockets between field stations and control room
Scheduling	Dynamically allocated, severity-ordered priority queue
Ordering Guarantee	Microsecond timestamp as secondary key for equal-severity messages
Coordination	Mutex + condition variable producer–consumer signalling
Process Isolation	Independent Message Processor via POSIX Named Pipe (FIFO)
Persistence	Mutex-protected, atomic append-only emergency log file
Validation	Bash-based stress test – 100 simultaneous client connections

01 Introduction & Objectives

Introduction

During disaster situations such as floods, earthquakes, industrial accidents and large-scale emergencies, the timely transmission and prioritization of information is critical. Conventional communication mechanisms may become inefficient when a large number of emergency reports are generated concurrently from geographically distributed field stations.

The Real-Time Disaster Communication System is a C-based, networked emergency communication platform designed to address this problem through concurrent client handling, priority-based message scheduling, inter-process communication and persistent logging.

The system follows a distributed architecture consisting of a **Central Control Room (Server)**, multiple **Field Stations (Clients)**, and an independent **Message Processor** process. Field stations transmit structured emergency messages to the control room using TCP sockets. The server concurrently receives these messages, assigns priority according to emergency severity and arrival time, and places them into a dynamically allocated priority queue. A dedicated dispatcher removes messages from the queue according to priority and forwards them to the independent Message Processor through a POSIX Named Pipe (FIFO). Simultaneously, every received message is persisted in an external log file, providing an auditable record of incoming emergency information.

> **CORE IDEA** Separate message reception, priority scheduling and message processing into concurrent, loosely-coupled components so that a burst of simultaneous emergency reports is never serialized behind a single slow operation.

Systems Concepts Applied

- TCP socket-based client–server communication
- POSIX threads (`pthreads`)
- Detached worker threads
- Dynamic memory allocation
- Linked-list-based priority queues
- Mutex-based synchronization
- Condition variables
- POSIX inter-process communication
- Named Pipes (FIFO)
- File-based persistent logging & Bash stress testing

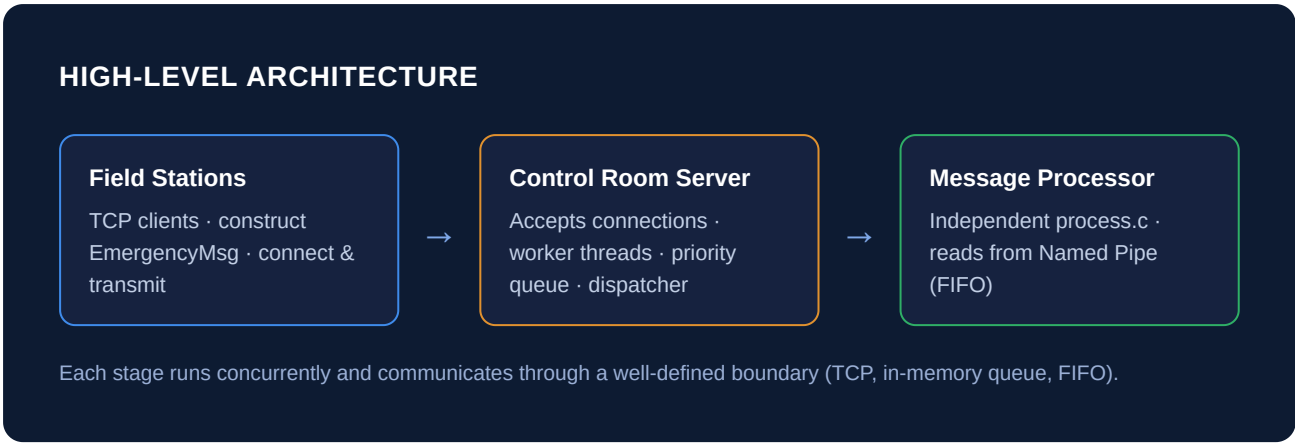
Project Objectives

OBJECTIVE	DESCRIPTION
Real-time communication	Enable field stations to transmit emergency reports to a centralized control room with minimal overhead
Concurrent client handling	Process multiple incoming client connections simultaneously without blocking
Priority-aware processing	Ensure high-severity emergency reports are processed before lower-severity reports
Deterministic ordering	Preserve FIFO ordering among messages of identical severity

Safe concurrent access	Prevent race conditions on shared message queues and log files
Process isolation	Separate message reception and processing using IPC
Reliable logging	Persist every received message immediately to an external log file
Stress tolerance	Validate reliability under 100 simultaneous client connections

02 System Architecture & Components

The system is organized into three primary components, connected in a linear pipeline that separates network communication, message scheduling and message processing into logically independent stages:



2.1 Field Station – Client

Each field station operates as a TCP client. When an emergency is detected, the client constructs an `EmergencyMsg` structure containing the emergency details, severity level and timestamp, then establishes a TCP connection with the Control Room Server and transmits the structured message through the socket. TCP provides reliable, connection-oriented communication, and multiple field stations can connect simultaneously.

2.2 Control Room – Central Server

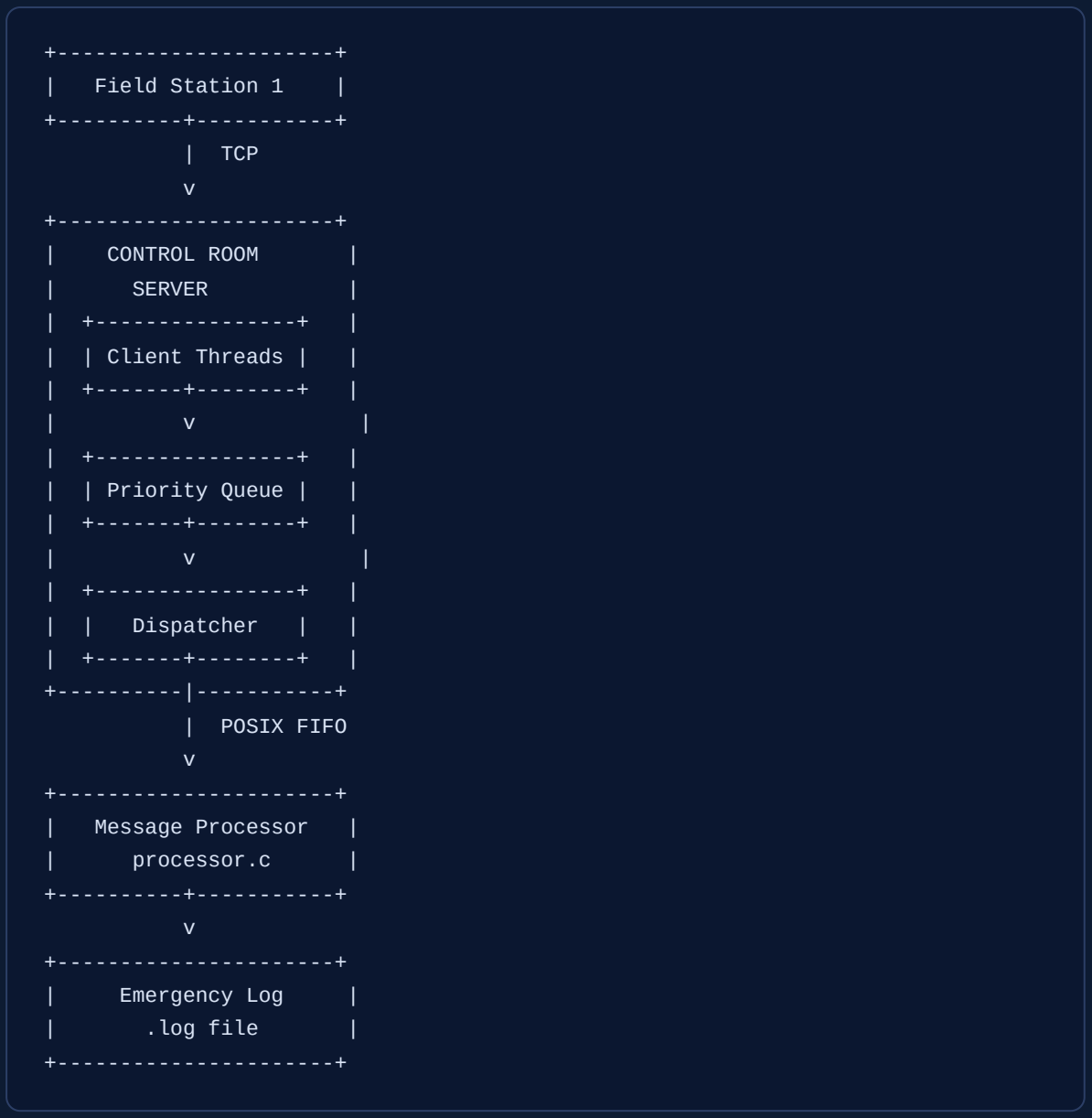
The server is the central coordination component. Its responsibilities include:

- Accepting incoming TCP connections
- Creating a worker thread for each connected client
- Receiving `EmergencyMsg` structures and recording arrival time
- Writing received messages to the log
- Inserting messages into the priority queue
- Synchronizing concurrent access to shared resources
- Dispatching messages to the processor

2.3 Message Processor

The Message Processor is implemented as a completely separate `processor.c` process rather than a function inside the server. It receives messages through a POSIX Named Pipe, giving process-level separation between message acquisition and downstream processing.

DETAILED COMPONENT FLOW



03 Priority Queue Design

The server implements the priority queue using a dynamically allocated linked list. Unlike a conventional FIFO queue, the insertion position depends on message priority.

Ordering Rules

- **Primary criterion:** Severity (levels 1–5, where 5 is most urgent)
- **Secondary criterion:** Microsecond-resolution timestamp, used when severity is equal

Dynamic allocation lets the queue grow with workload instead of requiring a fixed maximum capacity.

PRIORITY ORDERING

Severity 5
Highest urgency – served first

→

Severity 4–2
Ranked by severity

→

Severity 1
Lowest urgency – served last

For equal severity: earlier microsecond timestamp is dispatched before a later one, preserving chronological order.

TECHNICAL PRECISION Severity determines relative urgency; the microsecond timestamp is only a tie-breaker. It does not override severity ordering – a lower-severity message never overtakes a higher-severity one regardless of arrival time.

04 Concurrency & Synchronization

Concurrency is a fundamental requirement because emergency reports may originate from many field stations at approximately the same time. A sequential server would process one connection at a time, potentially delaying subsequent emergency messages. The system uses POSIX threads to overcome this limitation.

4.1 Client Worker Threads

Whenever the server accepts a new client connection, it creates a dedicated, **detached** worker thread that independently handles that client – receiving the message, recording its timestamp, logging it, acquiring the queue lock, inserting the message by priority, signalling the dispatcher, releasing the lock, and terminating. Because threads are detached, their resources are reclaimed automatically without an explicit `pthread_join()`.

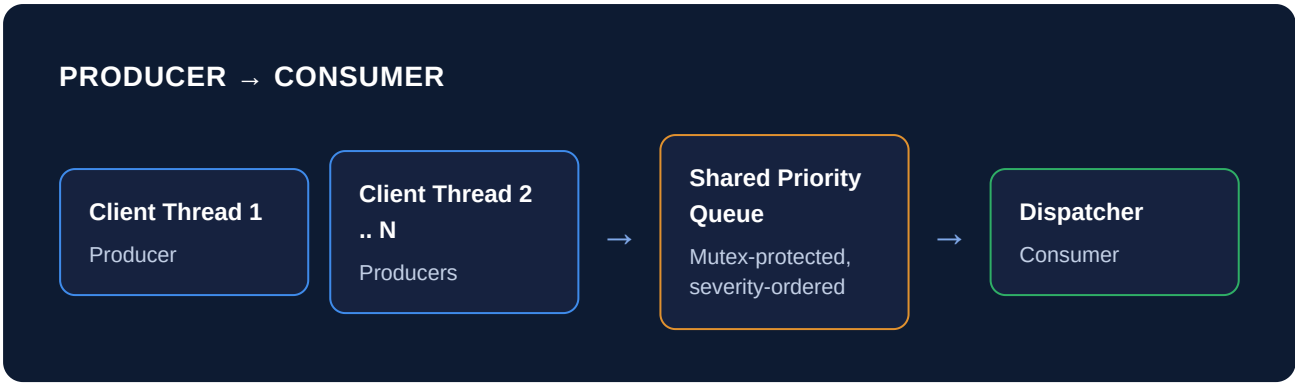
WORKER THREAD FAN-OUT

```
Server
  |
  +-- accept(Client 1) → Thread 1
  |
  +-- accept(Client 2) → Thread 2
  |
  +-- accept(Client 3) → Thread 3
  |
  +-- accept(Client N) → Thread N
```

05 Producer–Consumer Synchronization

5.1 Producer–Consumer Model

The concurrency architecture follows a classic producer–consumer pattern. Client-handling threads act as **producers** – they generate and insert messages into the shared priority queue. The dispatcher thread acts as the **consumer** – it removes messages from the queue and forwards them to the Message Processor.



5.2 Mutex Synchronization

Because multiple worker threads can attempt to modify the priority queue simultaneously, unrestricted access would cause race conditions – for example, two threads concurrently rewriting the same linked-list pointers and corrupting the list. A `pthread_mutex_t` protects the critical section covering queue insertion, queue removal and examination of queue state, so only one thread executes a protected queue operation at a time.

CORE FAILURE AVOIDED Thread A modifies the queue while Thread B modifies the queue at the same instant → race condition → possible corrupted linked list. The mutex makes this interleaving impossible.

5.3 Condition Variable

A `pthread_cond_t` condition variable coordinates producers and the dispatcher. If the queue is empty, the dispatcher does not poll continuously – it waits on the condition variable. When a client thread inserts a new message, it signals the condition variable and the dispatcher wakes to extract the highest-priority message.

This design is significantly more CPU-efficient than continuous polling, since the dispatcher only consumes CPU resources when useful work is available.

5.4 Thread-Safe Logging

The external log file is another shared resource. Since multiple client threads can receive messages simultaneously, concurrent file writes could interleave or corrupt log records. A separate mutex protects the logging operation so each received emergency message is recorded as an atomic logical entry.

ATOMIC LOG WRITE

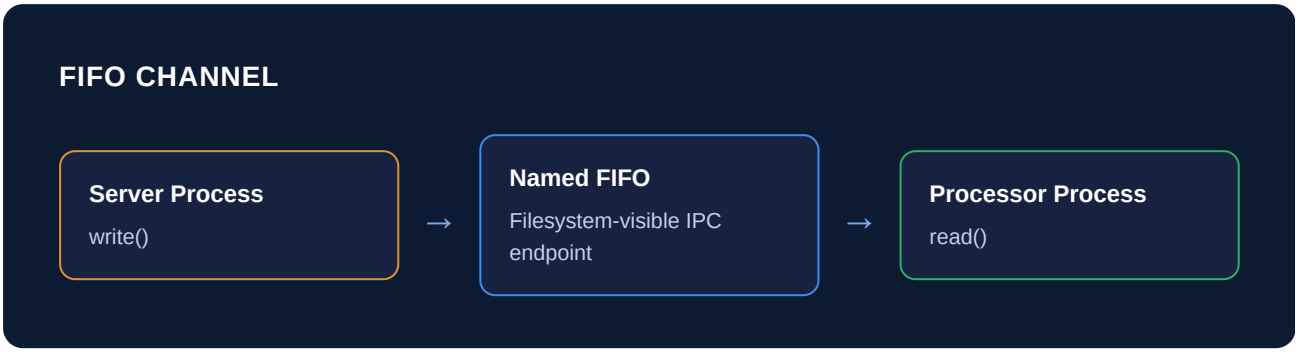
```
Receive Message → Acquire Log Mutex → Write Complete Record → Release Log Mutex
```

06 Inter-Process Communication

A major architectural feature of the project is the use of Inter-Process Communication (IPC) to connect the server with the independent Message Processor.

6.1 POSIX Named Pipe (FIFO)

The system uses a POSIX Named Pipe, commonly referred to as a FIFO (First-In, First-Out). Unlike an anonymous pipe, a named pipe exists as a filesystem-visible IPC endpoint and can be independently accessed by unrelated processes.



6.2 Dispatcher Thread

The server contains a dedicated dispatcher thread whose sole responsibility is to transfer queued messages to the processor:

1. Wait for a message to become available
2. Acquire the queue mutex
3. Remove the highest-priority message
4. Release the queue mutex
5. Write the message to the Named Pipe
6. Continue processing subsequent messages

Importantly, client threads do not communicate directly with the processor – the priority queue acts as an intermediary scheduling layer, separating message reception from message processing.

6.3 Advantages of the FIFO Architecture

BENEFIT	EXPLANATION
Process isolation	Processor failures or modifications do not require restructuring the networking layer
Modularity	The processor can be developed and tested independently
Loose coupling	The server does not need to know the processor's internal implementation
Sequential transfer	Messages move through a stream-oriented FIFO mechanism
Extensibility	The processor can later be replaced without changing client networking

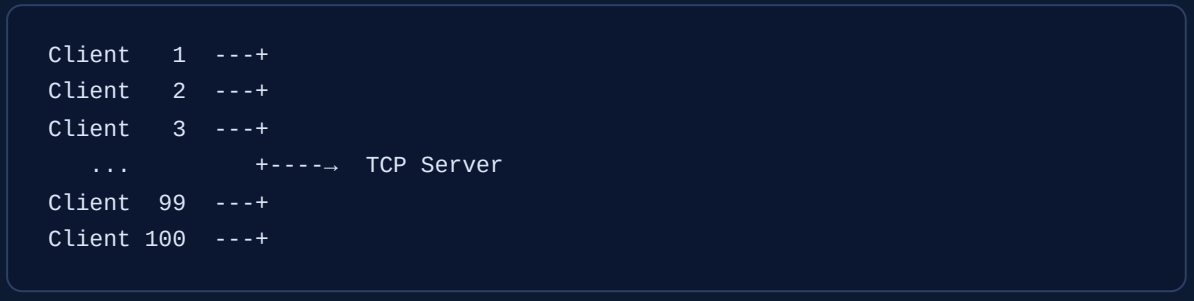
DESIGN PRINCIPLE Separation of concerns: the server is responsible for communication and scheduling, while the processor is responsible for downstream message handling.

07 Hackathon Challenge & Results

One of the most significant challenges addressed during the project was validating the system under simultaneous high-volume communication. Testing a single client does not adequately demonstrate the reliability of a concurrent communication system, so a Bash-based stress-testing mechanism was developed to launch **100 client connections simultaneously**.

7.1 Stress-Test Configuration

100-CLIENT CONCURRENCY STORM



The test placed simultaneous pressure on TCP handling, thread creation, dynamic memory allocation, priority-queue insertion, mutex synchronization, condition-variable signalling, file logging, dispatcher scheduling and Named Pipe communication.

7.2 Observed Results

TEST CRITERION	RESULT

Simultaneous clients	100
Client concurrency	Successfully handled
Message loss	0
Priority ordering	Preserved
Same-severity ordering	Preserved using timestamp
Queue synchronization	Successful
Concurrent logging	Successful
IPC transfer	Successful
Processing stability	Successful

The test particularly validated the correctness of the synchronization mechanism. Without proper mutex protection, simultaneous client threads could have corrupted the linked-list queue or produced inconsistent ordering; without synchronized logging, concurrent file writes could have resulted in incomplete or interleaved records. The successful 100-client stress test provided practical evidence that the concurrency model and synchronization strategy were functioning correctly.

RESULT The architecture maintained Concurrency + Priority + Synchronization + Persistence + IPC together within a single integrated C-based system, under a deliberately concurrent workload representative of a burst of emergency reports.

08 Limitations, Future Work & Conclusion

8.1 Limitations & Future Enhancements

- A production deployment would require authentication, authorization and encrypted transport such as TLS
- The current logging design would need hardened storage and crash-recovery semantics for production use
- The priority queue currently holds state in memory; a persistent database backend would improve durability
- Future work could add distributed control rooms, geographic emergency mapping, message acknowledgements, fault-tolerant processing, encrypted communication, and intelligent emergency prioritization

8.2 Key Outcomes

- Demonstrates low-level concurrent systems programming in C rather than hiding synchronization behind a high-level framework
- Detached worker threads allow the server to scale to many simultaneous field-station connections
- The severity + timestamp priority queue gives an explainable, deterministic scheduling policy
- Mutexes and condition variables together prevent race conditions while avoiding wasteful busy-waiting
- The POSIX Named Pipe cleanly separates message scheduling from message processing
- The 100-client stress test converts a design claim into measured, reproducible evidence

SYSTEM IN ONE SENTENCE The Real-Time Disaster Communication System shows that concurrency, priority scheduling, synchronization, persistence and process isolation can be combined into one reliable C-based emergency communication pipeline.

8.3 Conclusion

The Real-Time Disaster Communication System demonstrates the practical application of advanced C programming and operating-system concepts to a real-world emergency communication problem. The system integrates TCP socket networking, POSIX threads, dynamic memory allocation, linked-list data structures, priority scheduling, mutexes, condition variables, file synchronization and POSIX Named Pipe IPC into a cohesive architecture.

The central server provides concurrent communication capability by allocating a detached worker thread to each incoming client connection. The dynamically allocated priority queue ensures emergency messages are processed according to severity, while microsecond-resolution timestamps preserve chronological ordering among messages of equal severity. The producer–consumer synchronization model provides safe coordination between client threads and the dispatcher: mutexes prevent race conditions, while condition variables eliminate inefficient busy-waiting. Mutex-protected logging ensures incoming emergency reports are persistently recorded.

The use of a separate Message Processor connected through a POSIX Named Pipe further improves modularity and process isolation, demonstrating how complex systems can be decomposed into independent components while maintaining reliable communication between them. Most importantly, the successful stress test involving 100 simultaneous client connections – with zero message loss and correctly ordered processing – validates the effectiveness of the proposed architecture under concurrent workload conditions.

Overall, the project demonstrates that low-level C programming, combined with appropriate operating-system primitives and carefully designed synchronization mechanisms, can be used to construct a reliable, concurrent and extensible real-time communication infrastructure suitable for emergency-response scenarios.

Keywords: C11 • POSIX Threads • pthread_mutex_t • pthread_cond_t • TCP Sockets • Client–Server • Priority Queue • Producer–Consumer • POSIX Named Pipe (FIFO) • IPC • Persistent Logging • Disaster Communication